

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 2

Проектирование и технический дизайн API

Выполнил:
Прель Александр
Группа БР 1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

- 1 Описать все функциональные и нефункциональные требования к backend-приложению.
- 2 Выбрать формат реализации API: REST API или Backend For Frontend.
- 3 Спроектировать все эндпоинты API в формате OpenAPI-схемы с учетом реализованной диаграммы БД.
- 4 Описать тело каждого запроса и ответа, продумать возможные ошибки, возвращаемые из API.

Ход работы

На основе выбранного варианта приложения для бронирования столиков в ресторанах и схемы БД из ДЗ1 были сформированы требования и спроектирована REST API-спецификация в формате OpenAPI 3.0.3. Для документации Swagger подготовлен отдельный файл dz2.yaml.

Функциональные требования

- Регистрация пользователя по имени, фамилии, email, телефону и паролю.
- Вход пользователя по email и паролю с выдачей JWT.
- Получение данных текущего пользователя.
- Обновление профиля текущего пользователя.
- Получение списка ресторанов с фильтрацией по city, cuisine_id, price_category.
- Поиск ресторанов по части названия без учета регистра.
- Сортировка списка ресторанов по ценовой категории и среднему рейтингу.
- Поддержка пагинации для списка ресторанов.
- Получение детальной информации о ресторане.
- Получение меню ресторана по категориям.
- Получение списка отзывов ресторана с пагинацией.
- Добавление отзыва к ресторану авторизованным пользователем.
- Получение фотографий ресторана.
- Получение списка доступных кухонь.
- Получение истории собственных бронирований с пагинацией и фильтром по статусу.
- Создание бронирования по ресторану, дате, интервалу времени и числу гостей.
- Автоматический подбор подходящего активного столика на стороне сервера.
- Получение одного конкретного бронирования.
- Отмена собственного бронирования.

Нефункциональные требования

- Формат API: REST over HTTP/HTTPS, обмен данными в JSON, кодировка UTF-8.
- Формат дат и времени: дата YYYY-MM-DD, время HH:mm:ss, timestamps в ISO 8601.

- Авторизация защищенных методов через Bearer JWT.
- Пароли в БД хранятся только в виде password_hash.
- Единый формат ошибок для всех эндпоинтов.
- Валидация входных данных с возвратом 422 Unprocessable Entity.
- Корректные HTTP status codes: 200, 201, 400, 401, 403, 404, 409, 422, 500.
- Пагинация для списков ресторанов, отзывов и бронирований.
- Поиск и сортировка ресторанов выполняются на стороне сервера и совместимы с пагинацией.
- Создание бронирования должно выполняться атомарно с проверкой конфликта по времени.
- API должен быть логичным и достаточно простым для учебной реализации, без админских и избыточных служебных методов.

Перечень эндпоинтов

- 1 POST /auth/register - регистрация пользователя и немедленная выдача JWT. Параметры: нет. Body: RegisterRequest. Успех: 201 AuthResponse. Ошибки: 400, 409, 422, 500.
- 2 POST /auth/login - вход пользователя. Параметры: нет. Body: LoginRequest. Успех: 200 AuthResponse. Ошибки: 400, 401, 422, 500.
- 3 GET /users/me - получение профиля текущего пользователя. Параметры: нет. Body: нет. Успех: 200 User. Ошибки: 401, 404, 500.
- 4 PATCH /users/me - обновление профиля текущего пользователя. Параметры: нет. Body: UserUpdateRequest. Успех: 200 User. Ошибки: 400, 401, 409, 422, 500.
- 5 GET /restaurants - список ресторанов. Query: name, city, cuisine_id, price_category, sort_by, sort_order, page, limit. sort_by принимает значения price_category или rating, sort_order - asc или desc. Успех: 200 PagedRestaurantsResponse. Ошибки: 400, 500.
- 6 GET /restaurants/{restaurantId} - детальная карточка ресторана. Path: restaurantId. Body: нет. Успех: 200 RestaurantDetails. Ошибки: 404, 500.
- 7 GET /restaurants/{restaurantId}/menu - меню ресторана по категориям. Path: restaurantId. Body: нет. Успех: 200 MenuResponse. Ошибки: 404, 500.
- 8 GET /restaurants/{restaurantId}/reviews - отзывы ресторана. Path: restaurantId. Query: page, limit. Body: нет. Успех: 200 PagedReviewsResponse. Ошибки: 404, 500.
- 9 POST /restaurants/{restaurantId}/reviews - создание отзыва. Path: restaurantId. Body: ReviewCreateRequest. Успех: 201 Review. Ошибки: 400, 401, 403, 404, 422, 500.
- 10 GET /restaurants/{restaurantId}/photos - фотографии ресторана. Path: restaurantId. Body: нет. Успех: 200 RestaurantPhotosResponse. Ошибки: 404, 500.
- 11 GET /cuisines - справочник кухонь. Параметры: нет. Body: нет. Успех: 200 CuisinesResponse. Ошибки: 500.
- 12 GET /reservations/me - история бронирований текущего пользователя. Query: status, page, limit. Body: нет. Успех: 200 PagedReservationsResponse. Ошибки: 401, 500.
- 13 POST /reservations - создание бронирования. Body: ReservationCreateRequest. Успех: 201 Reservation. Ошибки: 400, 401, 404, 409, 422, 500.
- 14 GET /reservations/{reservationId} - получение одного своего бронирования. Path: reservationId. Body: нет. Успех: 200 Reservation. Ошибки: 401, 403, 404, 500.
- 15 PATCH /reservations/{reservationId}/cancel - отмена своего бронирования. Path: reservationId. Body: CancelReservationRequest. Успех: 200 Reservation. Ошибки: 400, 401, 403, 404, 409, 500.

Параметры поиска и сортировки ресторанов

- name - строка для поиска по части названия ресторана без учета регистра.
- sort_by=price_category - сортировка по ценовой категории в порядке budget, medium, premium.
- sort_by=rating - сортировка по среднему рейтингу, рассчитанному на основе отзывов ресторана.

- `sort_order` - направление сортировки: `asc` или `desc`; по умолчанию используется `desc`.

Вывод

В ходе работы были описаны требования к backend-приложению и подготовлена OpenAPI-спецификация REST API для сервиса бронирования столиков в ресторанах. В API предусмотрены регистрация и вход пользователя, работа с профилем, поиск и фильтрация ресторанов, сортировка по ценовой категории и рейтингу, просмотр меню, фотографий и отзывов, а также создание, просмотр и отмена бронирований. Спецификация может быть открыта в Swagger для проверки структуры запросов и ответов.